Cognition

AI Auto-Fix Pipeline

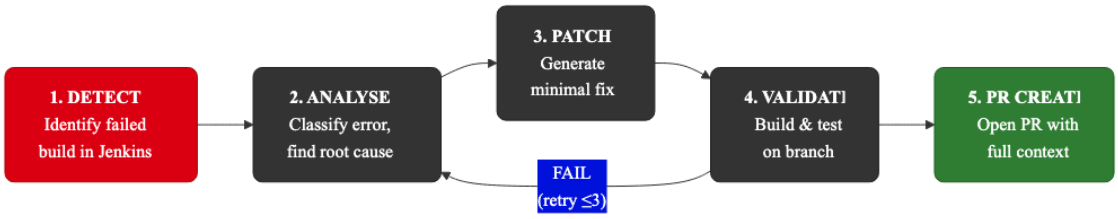
End-to-End Testing Guide

March 2026

About This Document

This guide walks you through end-to-end testing of the AI Auto-Fix Pipeline. The pipeline has **two modes** — one that uses MCP servers to connect directly to Jenkins, Confluence, GitHub and Nexus, and one where you paste data manually. Both modes follow the same five-stage flow:

Detect → Analyse → Patch → Validate → PR Create



This document covers both modes from start to finish, with test scenarios you can run to verify each stage works correctly.

What You Need

ITEM	MCP FLOW	MANUAL FLOW
Windsurf with project open	Required	Required
`.windsurf/rules/devops-pipeline.md` loaded	Required	Required
Jenkins MCP server connected	Required	Not needed
Confluence MCP server connected	Required	Not needed
GitHub MCP or `gh` CLI	Required	Optional
Nexus MCP server connected	Optional	Not needed
Jenkins build log (from `testing/scenarios/`)	Not needed	Required
Source files (from `testing/scenarios/`)	Not needed	Required

Test Scenarios

Four realistic failure scenarios are provided. Each includes all the data needed to test both flows end-to-end.

#	SCENARIO	ERROR TYPE	WHAT'S TESTED
1	Java compilation failure	`compilation`	Type mismatch after library upgrade. Two files to fix. Confluence known-issue cross-reference.
2	Node.js test failure	`test_failure`	Response format refactor broke 3 Jest tests. AI must fix the tests, not the source.
3	Dependency vulnerability	`dependency`	Nexus IQ blocked log4j-core (CVE-2021-44228). Fix is a `<dependencyManagement>` override.
4	K8s deployment failure	`deployment`	Two root causes: wrong image tag + memory request > limit. AI must fix both.

Each scenario lives in `testing/scenarios/XX-name/` and contains:

- `jenkins-console.log` — the full build failure output
- `source-files/` — the files that need fixing
- `confluence-known-issue.md` or `nexus-policy-report.json` — supplementary data (some scenarios)
- `expected-diagnosis.json` — what the AI should identify
- `expected-fix.diff` — the correct patch for comparison
- `README.md` — scenario description

Flow 1: Manual Mode (No MCP)

This is the recommended starting point. It works immediately — no infrastructure or MCP setup required. You paste data into Windsurf at each stage.

How to Start

In Windsurf, type:

```
@workflow auto-fix-manual
```

Windsurf loads the manual workflow and begins prompting you for data.

Stage 1: Failure Detection

What Windsurf asks: "Please paste the Jenkins build log or CI output."

What you do: Open the scenario's `jenkins-console.log` and paste the entire contents.

Example (Scenario 1):

```
testing/scenarios/01-java-compilation-failure/jenkins-console.log
```

What happens: The AI parses the log and extracts: - Job name and build number - Error type (compilation, test failure, dependency, deployment) - Affected files and line numbers

It will confirm: "I have identified a build failure in payment-service-build #247. The failure appears to be a compilation error. Should I proceed?"

You respond: Yes.

Stage 2: Log Analysis

What Windsurf asks: "Do you have any Confluence documentation or known-issue pages for this error?"

What you do (option A — provide data): Paste the contents of the scenario's supplementary file: - Scenario 1: `confluence-known-issue.md` - Scenario 3: `nexus-policy-report.json`

What you do (option B — skip): Type `skip` if the scenario has no supplementary data (Scenarios 2 and 4).

What happens: The AI produces a structured diagnosis: - Error type and root cause - Affected files with line numbers - Severity assessment - Suggested fix approach - Cross-reference with known issues (if Confluence data was provided)

Verify against: `expected-diagnosis.json` in the scenario folder.

Stage 3: Patch Generation

What Windsurf asks: "Please provide the source files that need fixing."

What you do: Paste the contents of each file in the scenario's `source-files/` directory.

Example (Scenario 1): Paste both: - `TransactionService.java` - `PaymentProcessor.java`

What happens: The AI generates a unified diff fixing the identified issues. It explains what was changed and why.

Verify against: `expected-fix.diff` in the scenario folder. The AI's diff should address the same root cause, though the exact implementation may vary.

Stage 4: Validation

What Windsurf asks: "Please validate the fix. You can apply the patch locally and run the build, then paste the result."

What you do: Respond with a simulated validation result:

SCENARIO	VALIDATION RESPONSE
1	`Build validated: mvn clean install — BUILD SUCCESS, 127 tests pass, 0 failures.`
2	`Tests pass: npm test — 50 tests, 50 passed, 0 failed.`
3	`Build validated: mvn clean install — BUILD SUCCESS, no Nexus policy violations.`
4	`Deployment validated: kubectl rollout status — deployment successfully rolled out, pod running.`

What happens: The AI confirms the fix is valid and proceeds to PR creation.

To test retry logic: Instead of confirming success, say "Build failed — new error: [describe something]". The AI should re-analyse with the new error context and generate a revised patch (up to 3 retries).

Stage 5: PR Creation

What Windsurf asks: "How would you like to create the PR?"

Options: 1. **Git CLI** — Windsurf provides the git commands to run 2. **GitHub CLI** — Windsurf runs `gh pr create` if available 3. **Manual copy-paste** — Windsurf generates the full PR title and body for you to copy

What you do: Choose option 3 for demo purposes.

What happens: The AI outputs a complete PR body containing: - Root cause analysis - Changes made and files modified - Risk assessment (scope, confidence, side effects) - Validation results - Original failure reference - Audit trail

That's the full manual flow, end to end.

Flow 2: MCP Mode (Automated)

This flow connects Windsurf directly to Jenkins, Confluence, GitHub and Nexus via MCP servers. The AI fetches data, creates branches, triggers builds, and opens PRs autonomously.

Setup (One-Time)

Before you can use MCP mode, the servers need to be installed and configured.

Option A — Run the setup script:

```
cd /path/to/devops-auto-fix-pipeline
bash scripts/setup-mcp.sh
```

This installs dependencies for each custom MCP server, prompts for your API credentials, creates a `.env` file, and offers to merge the MCP config into Windsurf.

Option B — Manual setup:

1. Install dependencies: `bash cd mcp-servers/jenkins-mcp && npm install cd ../confluence-mcp && npm install cd ../nexus-mcp && npm install`
2. Create `.env` in the project root: `JENKINS_URL=https://jenkins.internal.example.com JENKINS_USER=your-username JENKINS_TOKEN=your-api-token CONFLUENCE_URL=https://confluence.internal.example.com CONFLUENCE_USER=your-username CONFLUENCE_TOKEN=your-api-token NEXUS_URL=https://nexus.internal.example.com NEXUS_TOKEN=your-api-token GITHUB_PERSONAL_ACCESS_TOKEN=ghp_xxxxxxxxxxxx`
3. Merge `mcp-config.json` into your Windsurf MCP config: `~/.codeium/windsurf/mcp_config.json` Add the `jenkins`, `confluence`, `github`, and `nexus` server entries from the project's `mcp-config.json`. Do not overwrite — merge with your existing config.
4. Restart Windsurf to pick up the new MCP servers.

Verify MCP Servers Are Connected

In Windsurf, check that the MCP server indicators show the following servers as connected: - `jenkins` (or `jenkins-mcp`) - `confluence` (or `confluence-mcp`) - `github` - `nexus` (optional)

If any server is not connected, check the `.env` credentials and Windsurf logs.

How to Start

In Windsurf, type:

```
@workflow auto-fix-full
```

Stage 1: Failure Detection (Automated)

What happens: The AI calls `jenkins-mcp` → `get_failed_builds` to list recent failures across monitored jobs. No user input needed.

What you see: - The AI reports which jobs have failed builds - It selects the most recent/highest-priority failure - It calls `get_build_log` to retrieve the full console output

Output: Job name, build number, timestamp, and the raw build log — all fetched automatically.

Stage 2: Log Analysis (Automated)

What happens: The AI parses the build log to classify the error, then calls: - `confluence-mcp` → `search_known_issues` to find documented resolutions - `nexus-mcp` → `check_dependency_vulnerabilities` if it's a dependency issue

What you see: A structured diagnosis — same format as manual mode, but the AI gathered all the data itself.

Stage 3: Patch Generation (Automated)

What happens: The AI calls `github-mcp` (or uses `gh` CLI) to fetch the current source files from the repository. It generates a fix based on the diagnosis.

What you see: A unified diff with explanation. The AI may ask for confirmation before proceeding to apply.

Stage 4: Validation (Automated)

What happens: 1. The AI creates a feature branch: `ai-fix/{job-name}-{build-number}` 2. Applies the patch and commits 3. Calls `jenkins-mcp` → `trigger_build` on the feature branch 4. Polls `get_build_status` every 30 seconds (timeout: 15 minutes) 5. If the build passes → proceeds to Stage 5 6. If the build fails → re-analyses with the new error and retries (up to 3 attempts)

What you see: Build progress updates. On success, confirmation that all tests pass.

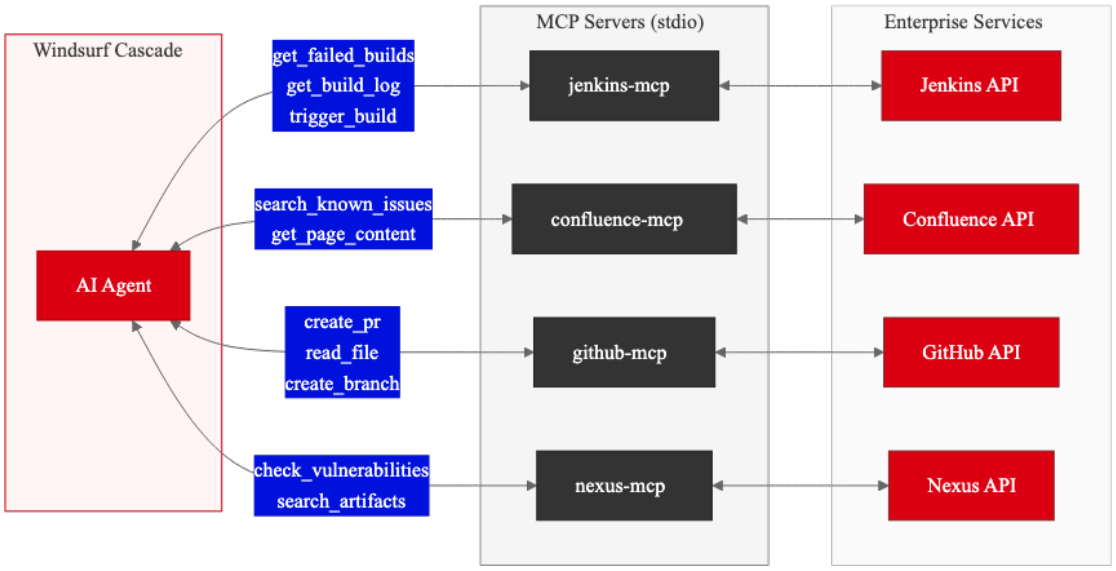
Stage 5: PR Creation (Automated)

What happens: The AI calls `github-mcp` or `gh pr create` to open a pull request with: - Title: `fix: {description} [auto-fix #{build-number}]` - Full PR body (root cause, changes, risk assessment, validation, audit trail) - Label: `ai-auto-fix` - Reviewers assigned based on CODEOWNERS

What you see: A PR URL. The pipeline is complete.

That's the full automated flow, end to end.

Side-by-Side Comparison



STAGE	MANUAL FLOW	MCP FLOW
1. Detect	You paste the Jenkins log	AI queries Jenkins MCP directly
2. Analyse	You paste Confluence/Nexus data (or skip)	AI searches Confluence + Nexus MCP
3. Patch	You paste source files	AI fetches files from GitHub MCP
4. Validate	You run the build and paste the result	AI triggers a Jenkins build and polls for result
5. PR Create	You choose git CLI, gh CLI, or copy-paste	AI creates branch + PR via GitHub MCP

Both flows produce the same outputs: a diagnosis, a patch, and a PR body. The difference is where the data comes from.

Running the Interactive Demo Script

For a guided walkthrough (manual flow), run:

```
bash testing/demo-runner.sh
```

This script: 1. Lets you select a scenario (1-4) 2. Displays the Jenkins log to paste at Stage 1 3. Shows Confluence/Nexus data for Stage 2 4. Provides source files for Stage 3 5. Shows the expected diagnosis and fix for comparison 6. Walks through validation and PR creation

Evaluation Checklist

Use this after each test run (either flow):

AREA	PASS CRITERIA
Detection	Correct job name, build number, error type extracted
Analysis	Root cause accurately identified, severity correct
Known issues	Confluence data used when provided (manual) or fetched (MCP)
Patch	Fix is correct, minimal, and addresses root cause
Validation	Appropriate validation steps (manual) or build triggered (MCP)
PR body	Contains root cause, changes, risk assessment, validation results
Mode	Pipeline operated correctly in the tested mode
Retry logic	On validation failure, AI retries with new context (max 3)
Security	No credentials or secrets in any output

Troubleshooting

ISSUE	SOLUTION
Windsurf doesn't recognise `@workflow`	Check `.windsurf/workflows/` directory exists and files have `.md` extension
AI doesn't follow the 5-stage flow	Ensure `.windsurf/rules/devops-pipeline.md` is loaded
MCP server won't connect	Run `bash scripts/setup-mcp.sh` and check `.env` credentials
AI generates wrong fix type	Provide more context — paste the full log, not just the error line
Jenkins MCP returns empty results	Check `JENKINS_URL` and credentials in `.env`. Ensure the Jenkins instance has recent failed builds.
Confluence search finds nothing	Check `CONFLUENCE_URL` and search space configuration. The known-issue must exist in an accessible space.
Validation build times out	Default timeout is 15 minutes. For large projects, consider increasing the poll timeout in the workflow.

Document version: 2.0 — March 2026 Project: AI-Driven DevOps Auto-Fix Pipeline